Documentation de validation

Erwan Didelot - Colin Dunand - Julia Mulder
Abel Prud'homme Lacroix - Baptiste Van Der Stracten

23 Janvier 2025

Contents

1 Introduction **1**

2 Descriptif des tests **1**
2.1 Réalisation des tests . . . 1
2.2 Organisation des tests . . . 2
2.2.1 Tests de la syntaxe . . . 2
2.2.2 Tests du contexte . . . 6
2.2.3 Tests de la génération de code . . . 9

3 Scripts de tests **11**
3.1 Automatisation des tests . . . 11
3.2 Limites de l'automatisation des tests . . . 12

4 Gestion des risques et gestion des rendus **12**
4.1 Risques spécifiques aux livrables . . . 12
4.1.1 Non-respect des formats ou délais des rendus . . . 12
4.1.2 Le compilateur ne passe pas le test fourni common-tests.sh . . . 12
4.1.3 Difficultés à réaliser l'analyse énergétique . . . 12
4.2 Risques spécifiques aux tests . . . 13
4.2.1 Base de tests incomplète . . . 13
4.3 Risques liés à la gestion des fichiers . . . 13
4.3.1 Désynchronisation entre branches Git, oubli de pull . . . 13
4.3.2 Scripts de test qui ne passent plus avec l'avancée du projet . . . 13

5 Couverture des tests (résultats de Jacoco) **14**

6 Annexe (scripts d'automatisation des tests) **17**

1 Introduction

Dans un projet informatique aussi complexe que le projet Génie Logiciel, la validation joue un rôle essentiel pour garantir la conformité, la qualité et la fiabilité du rendu final.

La validation de ce compilateur cherche à atteindre plusieurs objectifs :

- Il faut d'abord garantir que le compilateur est conforme aux spécifications en s'assurant que chaque étape du processus de compilation (analyse lexicale, analyse syntaxique, analyse contextuelle et génération de code) répond aux attentes définies par le cahier des charges.
- Il faut détecter et corriger les erreurs en identifiant le plus de problèmes de fonctionnement possibles, qu'ils soient liés à des bugs ou à des cas limites.
- Et enfin il faut assurer une couverture optimale en garantissant que les tests couvrent l'ensemble des fonctionnalités du compilateur ainsi que les cas d'usage courants et exceptionnels.

Ce document présente donc le processus de validation, avec les différents tests réalisés, les outils utilisés, et les résultats obtenus. De cette manière, les futurs contributeurs au projet pourront non seulement exécuter les tests actuels, mais aussi identifier les faiblesses potentielles de la couverture et proposer des améliorations.

2 Descriptif des tests

2.1 Réalisation des tests

Pour créer les tests, nous avons eu deux approches :

1. **La création de tests en "boîte noire"**

Les tests réalisés en "boîte noire" l'ont été sans connaissance interne du fonctionnement du compilateur. Cela permettait de s'assurer que le compilateur était conforme au cahier des charges lui-même. C'est notamment pour éviter d'écrire des tests qui seraient valides seulement pour le code que nous avons écrit et pas pour le compilateur attendu. La création des tests demande donc de porter une grande attention à la lexicographie, à la syntaxe concrète et à la syntaxe abstraite du langage Deca. Il faut les avoir bien comprises avant de commencer à créer des tests.

Pour organiser la création de ces tests, nous avons utilisé un tableau Google Sheets, avec une feuille pour chaque étape A, B ou C. Dans chacune de ces feuilles, on peut trouver un tableau recensant tous les tests réalisés, pour la partie sans objet et pour la partie objet. Pour chacun des tests, on a des informations sur s'il est valide ou invalide pour le compilateur, sur quelle partie de la grammaire ou du compilateur est testée et dans quel fichier .deca on peut trouver ce test.

2. La création de tests en "boîte blanche"

Les tests réalisés en "boîte blanche" l'ont été en se servant directement du code source du compilateur pour vérifier son comportement interne. Cela permettait de s'assurer que tous les chemins possibles dans le code sont correctement couverts et que les conditions internes sont respectées. C'est aussi à ce moment-là que l'outil Jacoco nous a été utile, ce qui sera développé dans la section "Couverture des tests" de ce document.

2.2 Organisation des tests

On peut trouver les tests de notre projet dans le répertoire `./src/test/deca`. Ils sont ensuite organisés dans trois répertoires différents qui correspondent aux 3 étapes A, B et C : `/syntax`, `/context` et `/codegen`.

2.2.1 Tests de la syntaxe

Les tests pour la vérification de la syntaxe u langage deca se trouvent dans le répertoire `/syntax`. Celui est organisé en deux répertoires : `/valid` et `/invalid` en fonction de si le test est censé être correct ou non pour notre parser. Dans chacun de ces répertoires, on trouve trois nouveaux répertoires : `/provided`, `/lexer` et `/parser`.

1. Tests dans `/syntax/valid/provided`

- `hello.deca` : test qui utilise println

2. Tests dans `/syntax/valid/lexer`

- `commentaires.deca` : test qui vérifie que le lexer reconnaît les différents types de commentaires
- `digit.deca` : test qui vérifie que le lexer reconnaît les chiffres
- `ident.deca` : test qui vérifie que le lexer reconnaît les identifiants
- `include.deca` : test qui vérifie que le lexer reconnaît l'inclusion de fichiers (si le fichier existe)
- `letter.deca` : test qui vérifie que le lexer reconnaît les lettres
- `litt_entiers.deca` : test qui vérifie que le lexer reconnaît les nombres entiers
- `litt_flottants.deca` : test qui vérifie que le lexer reconnaît les nombres flottants et leurs différentes écritures
- `reserved.deca` : test qui vérifie que le lexer reconnaît les mots réservés du langage (this, asm,...)
- `string.deca` : test qui vérifie que le lexer reconnaît les chaînes de caractères
- `symb_speciaux` : test qui vérifie que le lexer reconnaît les symboles spéciaux autorisés

3. Tests dans `/syntax/valid/parser`

- `and.deca` : test qui vérifie que le parser reconnaît l'opérateur "ET"
- `asm_multi_line_string.deca` : test qui vérifie que le parser reconnaît les méthodes écrites avec du code assembleur sur plusieurs lignes
- `asm_string.deca` : test qui vérifie que le parser reconnaît les méthodes écrites avec du code assembleur sur une seule ligne
- `assign_expr.deca` : test qui vérifie que le parser reconnaît l'assignement d'une expression à une variable
- `block.deca` : test qui vérifie que le parser reconnaît un bloc d'instructions
- `cast.deca` : test qui vérifie que le parser reconnaît le cast pour différentes types et objets
- `class_decl.deca` : test qui vérifie que le parser reconnaît la déclaration de classes
- `class_extension.deca` : test qui vérifie que le parser reconnaît l'extension de classes
- `commentaires.deca` : test qui vérifie que le parser reconnaît les commentaires, aussi les multi-lignes
- `decl_field.deca` : test qui vérifie que le parser reconnaît la déclaration de champs dans une classe
- `decl_var.deca` : test qui vérifie que le parser reconnaît la déclaration de variables
- `eq_neq.deca` : test qui vérifie que le parser reconnaît les opérateurs "==" et "!="
- `expr.deca` : test qui vérifie que le parser reconnaît les expressions complexes, avec des entiers, des flottants et des booléens
- `hello_world.deca` : test du "print("Hello World")" sur le parser
- `ident.deca` : test qui vérifie que le parser reconnaît l'identification de nouvelles variables
- `if_then_else.deca` : test qui vérifie que le parser reconnaît les branches if, else if et else
- `inequality.deca` : test qui vérifie que le parser reconnaît les opérateurs d'inégalités
- `inst.deca` : test qui vérifie que le parser reconnaît des instructions
- `instance_of.deca` : test qui vérifie que le parser reconnaît les instructions qui utilisent instanceof
- `instruct_vide.deca` : test qui vérifie que le parser reconnaît une instruction vide
- `list_decl_field.deca` : test qui vérifie que le parser reconnaît une déclaration de classe avec plusieurs champs (liste)
- `list_decl.deca` : test qui vérifie que le parser reconnaît une déclaration de plusieurs variables à la suite
- `list_expr.deca` : test qui vérifie que le parser reconnaît une liste d'expressions (ici dans un print)
- `list_inst.deca` : test qui vérifie que le parser reconnaît une liste d'instructions
- `list_decl_var.deca` : test qui vérifie que le parser reconnaît une déclaration de plusieurs variables à la suite
- `literal.deca` : test qui vérifie que le parser reconnaît les différents littéraux
- `main_vide.deca` : test qui vérifie que le parser reconnaît un bloc vide
- `modulo.deca` : test qui vérifie que le parser reconnaît l'opérateur modulo
- `mult.deca` : test qui vérifie que le parser reconnaît les opérateurs de multiplication et de division
- `new.deca` : test qui vérifie que le parser reconnaît la syntaxe de new (création d'un nouvel objet pour une classe)

- `null.deca` : test qui vérifie que le parser reconnaît l'utilisation de null
- `or.deca` : test qui vérifie que le parser reconnaît l'opérateur "OU"
- `param.deca` : test qui vérifie que le parser reconnaît l'utilisation de paramètres dans une méthode
- `print.deca`, `printlnx.deca`, `printx.deca`,: tests qui vérifient que le parser reconnaît l'utilisation de print, printlnx et printx (println est vérifié dans le test `hello_world.deca`)
- `readfloat.deca` et `readint.deca` : test qui vérifie que le parser reconnaît l'utilisation de readFloat() et readInt()
- `return.deca` : test qui vérifie que le parser reconnaît l'utilisation de return avec des expressions
- `select_expr.deca` : test qui vérifie que le parser reconnaît l'appel de méthodes sur des objets
- `sum.deca` : test qui vérifie que le parser reconnaît les opérateurs d'addition et de soustraction
- `this.deca` : test qui vérifie que le parser reconnaît l'utilisation de this dans une classe
- `type.deca` : test qui vérifie que le parser reconnaît les différents types pour une variable
- `unary.deca` : test qui vérifie que le parser reconnaît les opérations unaires (- et !)
- `visibility.deca` : test qui vérifie que le parser reconnaît la visibilité "protected" dans une classe
- `while.deca` : test qui vérifie que le parser reconnaît une boucle while

4. Tests dans `/syntax/invalid/provided`

- `simple_lex.deca` : test qui vérifie que le lexer n'accepte pas des instructions en dehors d'un bloc
- `chaine_incomplete.deca` : test qui vérifie que le lexer n'accepte pas de chaîne de caractère non fermée

5. Tests dans `/syntax/invalid/lexer`

- `accents.deca` : test qui vérifie que le lexer n'accepte pas de caractères accentués
- `arobase.deca` : test qui vérifie que le lexer n'accepte pas le caractère arobase (@)
- `esperluette.deca` : test qui vérifie que le lexer n'accepte pas le caractère arobase (&)
- `ident_1.deca` : test qui vérifie que le lexer n'accepte pas les identifiants avec des caractères invalides (comme #)
- `ident_2.deca` : test qui vérifie que le lexer n'accepte pas les identifiants avec des caractères invalides (comme les caractères accentués)
- `separateur .deca` : test qui vérifie que le lexer n'accepte pas les séparateurs comme caractère
- `string_1.deca` : test qui vérifie que le lexer n'accepte pas de chaîne de caractère non fermée
- `string_2.deca` : test qui vérifie que le lexer n'accepte pas de séparateur dans une chaîne de caractère (comme le saut de ligne)

6. Tests dans `/syntax/invalid/parser`

- `and.deca` : test qui vérifie que le parser n'accepte pas de mauvais opérateur (ici l'opérateur "ET" écris comme &

- `asm_faux_no_string.deca` : test qui vérifie que le parser n'accepte pas de méthode écrite en assembleur sans chaîne de caractère
- `assign_expr.deca` : test qui vérifie que le parser n'accepte pas la déclaration d'une variable avec un opérateur différent de =
- `block_faux.deca` : test qui vérifie que le parser n'accepte pas un bloc qui n'est pas fermé
- `cast.deca` : test qui vérifie que le parser n'accepte pas un cast écrit de la mauvaise manière (oubli de parenthèses)
- `class_sans_acc.deca` : test qui vérifie que le parser n'accepte pas la déclaration d'une classe qui n'est pas fermée
- `class_sans_acc.deca` : test qui vérifie que le parser n'accepte pas la déclaration d'une classe sans le mot "class" avant son nom
- `class_sans_corps.deca` : test qui vérifie que le parser n'accepte pas la déclaration d'une classe comme une instruction
- `class_sans_nom.deca` : test qui vérifie que le parser n'accepte pas la déclaration d'une classe sans lui donner de nom
- `commentaires.deca` : test qui vérifie que le parser n'accepte pas un commentaire pas fermé
- `decl_class_loop.deca` : test qui vérifie que le parser n'accepte pas la déclaration d'une classe à l'intérieur d'une déclaration de classe
- `decl_field.deca` : test qui vérifie que le parser n'accepte pas la déclaration fausse d'un champ dans une classe
- `decl_var_set_faux.deca` : test qui vérifie que le parser n'accepte pas la déclaration de plusieurs variables sans point virgule pour séparer les instructions
- `expr.deca` : test qui vérifie que le parser n'accepte pas une expression arithmétique binaire à laquelle il manque une opérande
- `extends_loop.deca` : test qui vérifie que le parser n'accepte pas l'utilisation du mot extends plus d'une fois pour la même classe
- `extension.deca` : test qui vérifie que le parser n'accepte pas l'utilisation du mot extends sans le nom de la classe que l'on veut étendre à la suite
- `float_faux.deca` : test qui vérifie que le parser n'accepte pas un float écrit de manière erronée
- `ident.deca` : test qui vérifie que le parser n'accepte pas une identifiant de variable qui commence par un chiffre
- `if_then_else_faux.deca` : test qui vérifie que le parser n'accepte pas l'utilisation d'un bloc if then else sans bloc d'instructions pour le else
- `if_then_else_faux_1.deca` : test qui vérifie que le parser n'accepte pas l'utilisation d'un bloc if then else sans parenthèses autour de la condition du if
- `include_1.deca` et `include_2.deca` : test qui vérifie que le parser n'accepte pas d'utilisation de include erronée
- `inst_faux_print_1.deca` et `inst_faux_print_2.deca` et leurs variante avec println, printx et printlnx : tests qui vérifient que le parser n'accepte pas d'utilisation de print sans parenthèses
- `instanceof.deca` : test qui vérifie que le parser n'accepte pas l'utilisation erronée de instanceof
- `list_decl_field.deca` : test qui vérifie que le parser n'accepte pas une déclaration erronée de champs dans une classe
- `list_expr_faux.deca` : test qui vérifie que le parser n'accepte pas une expression fausse

- `list_inst_faux.deca` : test qui vérifie que le parser n'accepte pas une instruction qui ne se finit pas par un point virgule
- `list_decl_var_faux.deca` : test qui vérifie que le parser n'accepte pas une liste fausse de déclarations de variables
- `literal_faux.deca` : test qui vérifie que le parser n'accepte pas un littéral faux
- `lvalue_xxx.deca` : tests qui vérifient que le parser n'accepte pas une lvalue fausse (booléenne, flottant, etc.)
- `main_decl_class.deca` : test qui vérifie que le parser n'accepte pas la déclaration d'une classe dans le bloc principal d'instructions
- `main_then_decl.deca` : test qui vérifie que le parser n'accepte pas la déclaration d'une classe après le bloc principal d'instructions
- `meth_sans_xxx.deca` : tests qui vérifient que le parser n'accepte pas la déclaration d'une méthode sans nom, sans type de retour, etc.
- `mot_reserve.deca` : test qui vérifie que le parser n'accepte pas l'utilisation d'un mot reservé par deca comme un identifiant
- `neq_faux.deca` : test qui vérifie que le parser n'accepte pas l'utilisation d'un opérateur faux pour l'inégalité
- `new.deca` : test qui vérifie que le parser n'accepte pas l'utilisation erronée de new
- `not_a_visibility.deca` : test qui vérifie que le parser n'accepte pas l'utilisation d'une visibilité qui n'existe pas
- `or_faux.deca` : test qui vérifie que le parser n'accepte pas l'utilisation d'un opérateur faux pour "OU"
- `param_sans_type.deca` : test qui vérifie que le parser n'accepte pas la déclaration d'une méthode avec un paramètre faux (il manque son type)
- `primary_expr_faux_X.deca` : tests qui vérifient que le parser n'accepte pas l'utilisation d'une primary_expr fausse (argument entre les parenthèses de new() et parenthèses pas ouvertes ou fermées notamment)
- `readInt.deca` et `readFloat.deca` : tests qui vérifient que le parser n'accepte pas d'argument entre les parenthèses de readInt() et readFloat()
- `visibility.deca` : test qui vérifie que l'attribut de visibilité est bien placé avant le type du champ de la classe

2.2.2 Tests du contexte

Les tests pour la vérification du contexte du langage deca se trouvent dans le répertoire `/context`. Celui-ci est organisé en deux répertoires : `/valid` et `/invalid` en fonction de si le test est censé être correct ou non pour notre parser. Dans chacun de ces répertoires, on trouve deux nouveaux répertoires : `/provided` et `/not_provided`.

1. **Tests dans** `/context/valid/provided`

- `hello-world.deca` : test qui vérifie le contexte pour println("Hello, world")

2. **Tests dans** `/context/valid/not_provided`

- `affect_bool.deca`, `affect_explicite.deca` et `affect_variable.deca` : tests qui vérifient le contexte pour l'affectation d'une valeur à une variable

- `arith_op.deca` : test qui vérifie la signature des opérateurs arithmétiques
- `binary_bool_op.deca` et `binary_op.deca` : tests qui vérifient le contexte pour la signature des opérateurs de comparaison ("EQ", "NEQ", etc.) et pour "ET" et "OU"
- `decl_class_main.deca`, `decl_class.deca` et `decl_class2.deca` : tests qui vérifient le contexte pour la déclaration d'une classe
- `decl_puis_affect.deca` : test qui vérifie le contexte pour la déclaration d'une variable sans initialisation puis son affectation ensuite
- `if_then_else.deca` : test qui vérifie le contexte pour le if then else
- `instanceof.deca` : test qui vérifie le contexte pour l'utilisation de instanceof
- `methode_call.deca` : test qui vérifie le contexte pour la déclaration d'une méthode puis son utilisation
- `mod_op.deca` : test qui vérifie le contexte pour l'opérateur modulo
- `new.deca` : test qui vérifie le contexte pour new
- `param.deca` : test qui vérifie le contexte pour la déclaration de plusieurs méthodes à l'intérieur d'une classe (avec la réutilisation des mêmes noms pour les paramètres des méthodes)
- `unary_op.deca` : test qui vérifie le contexte pour les opérateurs unaires (- et !)
- `while.deca` : test qui vérifie le contexte pour un while

On trouve aussi un répertoire `/not_working` qui contient des tests qui ne fonctionnent pas sur notre compilateur car la partie objet n'est pas entièrement implémentée pour les vérifications contextuelles mais qui doivent être vérifiés par un compilateur respectant le cahier des charges : ce sont des tests liés au cast, à l'extension de classe, à l'inclusion de fichiers et à l'utilisation du mot "this".

3. Tests dans `/context/invalid/provided`

- `affect-incompatible.deca` : test qui vérifie que la vérification contextuelle n'autorise pas une affectation de types incompatibles

4. Tests dans `/context/invalid/not_provided`

- `affect_apres_inst.deca` : test qui vérifie que la vérification contextuelle n'autorise pas la déclaration de variable après des instructions
- `affect_mauvais_type.deca` et `affect_mauvais_type2.deca` : tests qui vérifient que la vérification contextuelle n'autorise pas une affectation de types incompatibles
- `affect_meme_nom.deca` : test qui vérifie que la vérification contextuelle n'autorise pas la déclaration de deux variables avec le même nom
- `affect_sans_ident.deca` et `affect_sans_type.deca`: tests qui vérifient que la vérification contextuelle n'autorise pas la déclaration d'variable sans nom ou sans type
- `affect_string.deca` : test qui vérifie que la vérification contextuelle n'autorise pas la déclaration d'une variable de type string
- `affect_var_non_decla.deca` : test qui vérifie que la vérification contextuelle n'autorise pas l'utilisation d'une variable qui n'a pas été déclarée auparavant
- `cast.deca` et tests derivés: tests qui vérifient que la vérification contextuelle n'autorise pas d'utilisation fausse du cast

- `class_meme_nom.deca` : test qui vérifie que la vérification contextuelle n'autorise pas la déclaration de deux classes avec le même nom
- `decl_class_nexiste_pas.deca` : test qui vérifie que la vérification contextuelle n'autorise pas l'utilisation d'une classe qui n'a pas été déclarée
- `decl_field.deca` : test qui vérifie que la vérification contextuelle n'autorise pas une déclaration de champ fausse dans une classe
- `decla_void.deca` : test qui vérifie que la vérification contextuelle n'autorise pas une déclaration de variable de type void
- `plus_int_string.deca`, `moins_float_bool.deca`, `mult_string_bool.deca`, etc. : tests qui vérifient que la vérification contextuelle n'autorise pas l'utilisation d'un opérateur arithmétique avec des types non autorisés `et_int_bool.deca`, `ou_float_bool.deca`, etc. : tests qui vérifient que la vérification contextuelle n'autorise pas l'utilisation d'un opérateur binaire avec des types non autorisés
- `extends_apres.deca` : test qui vérifie que la vérification contextuelle n'autorise pas l'extension d'une classe par une classe qui n'a pas encore été déclarée
- `extends_attrib.deca` : test qui vérifie que la vérification contextuelle n'autorise pas l'utilisation d'un attribut pour une classe qui n'est pas censée l'avoir
- `extends_loop.deca` : test qui vérifie que la vérification contextuelle n'autorise pas l'extension d'une classe par elle-même `geq_bool_float.deca`, `leq_bool_bool.deca`, etc. : tests qui vérifient que la vérification contextuelle n'autorise pas l'utilisation d'un opérateur de comparaison avec des types non autorisés
- `inversion.deca` : test qui vérifie que la vérification contextuelle n'autorise pas un appel de méthode erroné
- `main_return.deca` : test qui vérifie que la vérification contextuelle n'autorise pas un return dans le bloc principal d'instructions
- `methode_call_param.deca`, `meth_non_decl.deca`, et autres tests dérivés sur les méthodes : tests qui vérifient que la vérification contextuelle n'autorise pas l'appel ou la déclaration de méthodes de manière erronée
- `modulo.deca` : test qui vérifie que la vérification contextuelle n'autorise pas l'utilisation de modulo avec des types autres que des entiers
- `new.deca` : test qui vérifie que la vérification contextuelle n'autorise pas l'utilisation de new avec une classe non déclarée
- `not_int.deca` : test qui vérifie que la vérification contextuelle n'autorise pas l'utilisation de l'opérateur unaire "!" sur autre chose que des booléens
- `param_type_diff.deca` : test qui vérifie que la vérification contextuelle n'autorise pas l'appel d'une méthode avec les mauvais type comme paramètres
- `print_bool.deca` : test qui vérifie que la vérification contextuelle n'autorise pas l'utilisation de print sur un booléen
- `protected.deca` : test qui vérifie que la vérification contextuelle n'autorise pas l'utilisation d'un attribut protected en dehors de sa classe
- `readFloat.deca` : test qui vérifie que la vérification contextuelle n'autorise pas l'attribution du résultat de readFloat() à autre chose q'un float
- `this_main.deca` : test qui vérifie que la vérification contextuelle n'autorise pas l'utilisation de this dans le bloc d'instructions principal
- `this.deca` : test qui vérifie que la vérification contextuelle n'autorise pas l'utilisation de this pour un attribut qui n'existe pas pour la classe

- `type_decla_faux.deca` : test qui vérifie que la vérification contextuelle n'autorise pas la déclaration d'une variable avec un type qui n'existe pas
- `while.deca` : test qui vérifie que la vérification contextuelle n'autorise pas l'utilisation de while avec dans sa condition autre chose qu'un booléen

2.2.3 Tests de la génération de code

Les tests de la génération de code se trouvent dans le répertoire `/codegen`. Celui-ci est organisé en quatre répertoires : `/valid`, `/invalid`, `/interactive` et `/perf`.

1. **Tests dans** `/codegen/valid/provided`

- `cond0.deca` : test qui vérifie la génération de code pour une condition dans un if then else
- `ecrit0.deca` : test qui vérifie la génération de code pour l'écriture avec print
- `entier1.deca` : test qui vérifie la génération de code pour la déclaration d'une variable, sa modification et son affichage
- `exdoc.deca` : test qui vérifie la génération de code pour la déclaration d'une méthode et son utilisation
- `exmathdoc.deca` : test qui vérifie la génération de code pour l'utilisation de la classe Math.deca (extension Trigo)

2. **Tests dans** `/codegen/valid/not_provided`

- `addition_6_variables.deca` : test qui vérifie la génération de code pour une addition avec 6 variables en même temps
- `aucun_debordement_int.deca` : test qui vérifie la génération de code pour une opération arithmétique qui dépasse l'entier maximal que l'on peut coder (pas d'erreur car pour les entiers les opérations se font modulo 32 bits)
- `binary_expr.deca` : test qui vérifie la génération de code pour les opérateurs binaires de comparaison
- `calcul_var_int_float.deca` : test qui vérifie la génération de code pour des calculs qui mélangent entiers et flottants (test la conversion d'entiers en flottants)
- `calcul_variables.deca` : test qui vérifie la génération de code pour des calculs avec dofférents types d'opérateurs arithmétiques
- `comp.deca` : test qui vérifie la génération de code pour les opérateurs de comparaison
- `cond.deca` : test qui vérifie la génération de code pour les conditions pour des boucles while
- `conv_float_div.deca` : test qui vérifie la génération de code pour une division entre des entiers et flottants
- `conv_float.deca` : test qui vérifie la génération de code pour une expression entre des entiers et flottants
- `decla_var.deca` : test qui vérifie la génération de code pour différentes déclarations de variables
- `factorielle_iter.deca` : test qui vérifie la génération de code pour le calcul de !10 itérativement
- `if_2elsif.deca` : test qui vérifie la génération de code pour une branche conditionnelle avec un if et deux elsif
- `if_3elsif_else.deca` : test qui vérifie la génération de code pour une branche conditionnelle avec un if, trois elsif et un else

- `if_else_imb.deca` : test qui vérifie la génération de code pour une branche conditionnelle avec un if imbriqué dans un autre if
- `if_else.deca` : test qui vérifie la génération de code pour une branche conditionnelle avec un if et un else
- `if_elsif_else.deca` : test qui vérifie la génération de code pour une branche conditionnelle avec un if, un elsif et un else
- `init_var.deca` : test qui vérifie la génération de code pour l'initialisation explicite d'une variable avant son utilisation
- `init_var.deca` : test qui vérifie la génération de code pour une ligne d'instruction vide
- `not.deca` : test qui vérifie la génération de code pour l'opérateur unaire ! dans une condition
- `oparith.deca` : test qui vérifie la génération de code pour des opérateurs arithmétiques
- `opbool.deca` : test qui vérifie la génération de code pour les opérateurs "ET" et "OU"
- `ordre_eval_et.deca` : test qui vérifie la génération de code pour l'ordre d'évalutation dans l'expression booléenne "ET"
- `ordre_eval_ou.deca` : test qui vérifie la génération de code pour l'ordre d'évalutation dans l'expression booléenne "OU"
- `print.deca` : test qui vérifie la génération de code pour les print
- `printx.deca` : test qui vérifie la génération de code pour les printx et printlnx (le résultat obtenu n'est pas correct car nous n'avons pas implémenté l'affichage des flottants sous forme hexadécimale)
- `unary_minus.deca` : test qui vérifie la génération de code pour l'opérateur unaire -
- `while_imb.deca` : test qui vérifie la génération de code pour deux boucles while imbriquées
- `while.deca` : test qui vérifie la génération de code pour une boucle while

On trouve aussi un répertoire `/not_working` qui contient des tests pour tester les cast, les instanceof, les déclarations de classes, l'utilisation de this et null, la déclaration de méthode. Ce sont des tests qui ne peuvent pas être vérifiés sur notre compilateur mais qui sont censés marcher sur un compilateur respectant le cahier des charges.

2. Tests dans `/codegen/invalid`

- `debordement_float.deca` : test qui vérifie l'erreur de génération de code pour une opération avec débordement arithmétique
- `division_zero.deca` : test qui vérifie l'erreur de génération de code pour une opération avec division par 0 (0 obtenu avec un calcul)
- `division_zero_float.deca` : test qui vérifie l'erreur de génération de code pour une opération flottante avec division par 0.0
- `division_zero_int.deca` : test qui vérifie l'erreur de génération de code pour une opération entière avec division par 0
- `init_cond.deca` : test qui vérifie l'erreur de génération de code pour l'utilisation d'une variable non initialisée
- `modulo_zero.deca` : test qui vérifie l'erreur de génération de code pour un modulo par 0
- `var_non_init_if.deca` : test qui vérifie l'erreur de génération de code pour l'utilisation d'une variable non initialisée
- `var_non_init.deca` : test qui vérifie l'erreur de génération de code pour l'utilisation d'une variable non initialisée

2. Tests dans `/codegen/interactive`

- `readFloat.deca` : test qui vérifie la génération de code pour readFloat() d'une variable non initialisée
- `readInt.deca` : test qui vérifie la génération de code pour readInt()

2. Tests dans `/codegen/perf`
Ce sont des tests qui servent à vérifier les performances des programmes générés en assembleur.

- `ln2_fct.deca` : test qui vérifie la génération de code pour le calcul de racine d'un polynôme par dichotomie dans une méthode
- `ln2.deca` : test qui vérifie la génération de code pour le calcul de racine d'un polynôme par dichotomie dans le bloc d'instructions principal
- `syracuse42.deca` : test qui vérifie la génération de code pour le calcul du temps de vol pour 42 de la suite de Syracuse

3 Scripts de tests

3.1 Automatisation des tests

La réalisation des tests a pu être automatisée dans le fichier pom.xml. Avec la commande `mvn test`, on lance tous les tests, ce qui va utiliser les scripts présentés ici. On peut trouver le code de ces scripts en annexe.

Pour les tests du lexer, du parser et du contexte, on utilise les tests fournis avec le projet dans le répertoire `./src/test/script/launchers`, ces scripts sont respectivement `test_lex`, `test_synt` et `test_context`.

Pour utiliser ces tests, on a pu créer respectivement les scripts `avance-lex.sh`, `avance-synt.sh` et `avance-cont.sh`. Ces trois scripts fonctionnent de la même manière, ils parcourent les repértoires des tests valides et invalides qui sont associés à la partie du compilateur que l'on teste. Si les tests sont censés être valides, le message "Succès attendu de test_[lex, synt, context] sur [nom_du_test.deca]" s'affiche, sinon c'est le message "Echec inattendu de test_[lex, synt, context] sur [nom_du_test.deca]" qui s'affiche, et le programme s'arrête. Si les tests sont censés être invalides, le message "Echec attendu de test_[lex, synt, context] sur [nom_du_test.deca]" s'affiche, sinon c'est le message "Succès inattendu de test_[lex, synt, context] sur [nom_du_test.deca]" qui s'affiche, et le programme s'arrête.

Pour éviter que le programme ne se bloque dès qu'un test ne passe pas, on peut commenter les lignes "exit 1" dans les différents scripts. Et pour ne récuperer que les tests qui ne renvoient pas le résultat attendu, il suffit d'utiliser `grep` par exemple avec la commande `mvn test | grep "inattendu"`.

Quant à l'automatisation pour la génération de code on utilise le script `avance-gencode.sh`. Pour cela, nous avons créé un répertoire `/expected` dans le dossier `test/deca/codegen/valid/`. Dans `/expected` on trouve des fichiers de type `nom_du_test.expected`, qui contiennent le résultat que l'on est censé obtenir lors de l'exécution du fichier .ass obtenu lors de la génération du code en assembleur pour les différents tests. Pour un test donné, l'automatisation génére donc le fichier assembleur .ass associé, l'exécute avec ima, stocke ce résultat dans un fichier .out et compare le fichier.out au fichier associé .expected. Si le résultat est le même, le message "Test réussi pour nom_du_test" s'affiche, sinon c'est le message "Test échoué pour nom_du_test". Dans ce cas, le résultat attendu et le résultat obtenu s'affichent et le script s'arrête. Si tous les tests ont réussi, le message "Tous les tests de gencode ont été exécutés avec succès" s'affiche.

Lors de l'exécution automatique des tests, le script `common-tests.sh` est aussi lancé.

Ces automatisations sont utiles pour tester tout au long du projet quelles parties du compilateur fonctionnent, d'où la nécessité de réaliser ces tests avant même de commencer à coder (tests en boîte noire). Elles servent aussi à s'assurer tout au long du projet que les différentes modifications que l'on fait sur les fichiers ne font pas buguer des tests qui passaient auparavant.
L'outil Jacoco nous permet de nous assurer que nos tests couvrent suffisamment de parties de notre code, son utilisation et ses résultats seront présentés dans la section "Couverture des tests (résultats de Jacoco)".

3.2 Limites de l'automatisation des tests

Nos tests ne sont pas automatisés par rapport à la vérification des arbres obtenus après l'étape A et l'étape B. Cela a pu poser problème car les tests test_synt et test_context pouvaient être validées pour certains tests alors que les arbres obtenus n'étaient pas bons. De plus, nous n'avons pas non plus effectué de tests pour la vérification automatique des options de decac. Nous avons donc réalisé des vérifications manuelles pour la plupart des tests tout au long du projet pour s'assurer qu'il n'y avait pas de problèmes.

4 Gestion des risques et gestion des rendus

4.1 Risques spécifiques aux livrables

4.1.1 Non-respect des formats ou délais des rendus

Les documents rendus pourraient ne pas respecter les formats ou les échéances exigés.

Niveau de risque : Élevé

Actions :

- Mettre en place un planning clair et détaillé des étapes et dates de rendus.
- Prévoir qu'une personne qui n'a pas travaillé sur le document le relise avant le rendu pour avoir un œil extérieur.

4.1.2 Le compilateur ne passe pas le test fourni common-tests.sh

Le compilateur ne passe pas le test fourni common-tests.sh.

Niveau de risque : Élevé

Actions :

- Vérifier régulièrement que le compilateur passe ce script de tests.
- Tester chaque composante avant l'intégration finale.

4.1.3 Difficultés à réaliser l'analyse énergétique

L'analyse énergétique pourrait ne pas être complète ou rigoureuse en raison d'un manque de données ou d'expertise.

Niveau de risque : Moyen

Actions :

- Recueillir des données dès le début du projet.
- Allouer un temps dédié à l'analyse énergétique en parallèle des autres tâches.

4.2 Risques spécifiques aux tests

4.2.1 Base de tests incomplète

Tests essentiels Il est essentiel de tester le compilateur sur certains cas qui empêcheraient n'importe quel programme de compiler. Notamment la reconnaissance du point-virgule comme la fin d'une instruction, des accolades ouvrantes et fermantes pour la démarcation de bloc, la déclaration d'une variable, d'une classe et de méthodes, etc.

Niveau de risque : Élevé

Actions :

- Tester au moins chacun de ces cas sur toutes les étapes (lexer, parser, contexte, codegen)

Tests sur les opérateurs binaires Il est impossible de tester toutes les combinaisons d'opérateurs binaires (+, -, `&&`, `||`, etc.) et de types (int, float, bool). Il peut donc y avoir un risque que certaines combinaisons ne fonctionnent pas. On peut cependant raisonnablement supposer que si l'on teste au moins une fois chaque opérateur et type, on limite le risque d'erreurs du compilateur.

Niveau de risque : Moyen

Actions :

- Tester au moins chacun des cas sur toutes les étapes (lexer, parser, contexte, codegen)

Tests sur les caractères interdits (lexer) On ne peut pas tester le lexer sur tous les caractères interdits (caractères accentués, arobase, etc.).

Niveau de risque : Moyen

Actions :

- Tester au moins des caractères accentués car c'est majoritairement ceux que l'on risque de voir apparaître par erreur dans les noms de variable ou de classe.

4.3 Risques liés à la gestion des fichiers

4.3.1 Désynchronisation entre branches Git, oubli de pull

Des modifications peuvent avoir été réalisées sur une branche locale, mais ne pas avoir été fusionnées dans la branche principale. On peut aussi oublier de récupérer les modifications faites par d'autres membres de l'équipe et qui ont été push sur le git. Cela peut entraîner un dysfonctionnement du projet.

Niveau de risque : Élevé

Actions :

- Récuperer toutes les modifications de fichiers qui ont été push avant de commencer à coder.
- Réaliser régulièrement des git status.
- Tester régulièrement le contenu du dépot partagé, en plus des contenus des dépots locaux.

4.3.2 Scripts de test qui ne passent plus avec l'avancée du projet

L'avancée des différentes étapes du projet ou encore l'ajout de la partie objet ou de l'extension pourraient avoir pour conséquence que certains tests qui fonctionnaient auparavant posent problème. Il faut vérifier régulièrement que les modifications faites au fur et à mesure ne créent pas de dysfonctionnements sur des étapes qui fonctionnaient avant.

Niveau de risque : Élevé

Actions :

- Tester très régulièrement la base de tests sur toutes les étapes pour s'assurer du fonctionnement de tous les tests tout au long du projet.

5 Couverture des tests (résultats de Jacoco)

La couverture des tests est utilisée pour évaluer dans quelle mesure le code est suffisament testé par nos tests. Jacoo est un outil qui permet de mesurer cette couverture en identifiant les lignes et les branches de code exécutées lors des tests.

On peut considérer qu'une bonne couverture a un pourcentage supérieur à 70%.

Dans le cadre de notre projet, on peut obtenir le rapport Jacoco avec la commande `mvn -Djacoco.skip=false verify`. L'ensemble des scripts d'automatisation qui ont été ajoutés au fichier pom.xml vont être exécutés et on obtient un fichier `target/site/jacoco/index.html` que l'on peut ouvrir au moyen d'un navigateur.

Après exécution de cette commande, on obtient pour notre projet une couverture égale à 72%.

Deca Compiler

Element	Missed Instructions	Cov.	Missed Branches	Cov.
fr.ensimag.deca.codegen		96 %		75 %
fr.ensimag.deca.context		76 %		55 %
fr.ensimag.deca.syntax		74 %		56 %
fr.ensimag.deca.tree		73 %		63 %
fr.ensimag.ima.pseudocode		68 %		75 %
fr.ensimag.deca		52 %		38 %
fr.ensimag.ima.pseudocode.instructions		48 %		n/a
fr.ensimag.deca.tools		38 %		33 %
Total	5 613 of 20 451	72 %	484 of 1 154	58 %

Figure 1: Rapport Jacoco

On voit donc que les tests ne parcourent pas 5613 instructions sur 20 451. Cela est notamment expliqué par le fait qu'il n'y a pas d'automatisation de test pour les fonctions de décompilation mais celles-ci ont été testées manuellement avec l'option decac -p. On peut considérer que nos tests couvrent bien notre code puisqu'on a une couverture de 72% ce qui est supérieur aux 70% attendus.

On remarque cependant que la couverture des branches n'est que de 58%. La couverture des branches est celle des branches des instructions conditionnelles. Pour améliorer la base de tests, on aurait donc pu couvrir plus de cas dans les instructions conditionnelles. Quand on regarde la couverture des branches pour le package fr.ensimag.deca.tree plus en détail (classés par couverture des branches dans l'ordre décroissant dans les figures 2 et 3 ci-dessous), on se rend compte que la plupart des branches non-couvertes se trouvent dans des fichiers liés à la partie avec objet que nous avons moins traitée par manque de temps. On peut donc conclure que concernant la partie sans-objet, nous avons une bonne couverture avec nos tests.

fr.ensimag.deca.tree

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
BooleanLiteral		84 %		100 %	2	9	3	14	2	8	0	1
TreeList		93 %		100 %	1	14	1	21	1	11	0	1
Plus		97 %		100 %	1	6	1	18	1	3	0	1
Minus		97 %		100 %	1	6	1	19	1	3	0	1
Multiply		97 %		100 %	1	6	1	19	1	3	0	1
UnaryMinus		96 %		100 %	1	6	1	16	1	4	0	1
Not		96 %		100 %	1	5	1	13	1	4	0	1
AbstractOpBool		100 %		100 %	0	4	0	8	0	2	0	1
AbstractOpArith		100 %		95 %	1	15	0	38	0	5	0	1
AbstractOpCmp		96 %		90 %	3	22	1	64	0	3	0	1
DeclVar		84 %		87 %	2	11	7	50	1	7	0	1
Divide		91 %		87 %	2	8	2	23	1	4	0	1
Tree		69 %		81 %	8	30	20	69	6	22	0	1
AbstractPrint		63 %		75 %	4	15	9	29	2	9	0	1
Assign		88 %		75 %	4	11	4	35	1	5	0	1
Modulo		86 %		75 %	3	8	2	22	1	4	0	1
Identifier		57 %		68 %	9	31	24	80	3	20	0	1
AbstractExpr		68 %		66 %	7	19	10	37	4	13	0	1
ListInst		68 %		66 %	2	8	5	15	1	5	0	1
MethodCall		66 %		64 %	7	16	18	46	4	9	0	1
ListDeclParam		66 %		60 %	6	14	10	25	4	9	0	1
ListDeclClass		63 %		60 %	4	12	9	27	2	7	0	1
ListDeclField		68 %		57 %	7	18	7	30	3	11	0	1
NoInitialization		47 %		50 %	5	13	11	25	1	7	0	1
DeclParam		66 %		50 %	10	15	12	32	5	10	0	1
ListDeclMethod		55 %		50 %	9	17	12	30	5	10	0	1
Cast		67 %		50 %	5	9	11	23	3	7	0	1
AbstractBinaryExpr		68 %		50 %	2	9	6	27	1	8	0	1
ListDeclVar		56 %		50 %	3	8	9	20	1	4	0	1
New		55 %		50 %	3	7	7	17	2	6	0	1

Figure 2: Rapport Jacoco pour fr.ensimag.deca.tree

Initialization		63 %		50 %	4	9	8	21	3	8	0	1
Return		64 %		50 %	5	10	8	19	3	7	0	1
FloatLiteral		84 %		50 %	4	12	4	25	1	9	0	1
ThisLiteral		54 %		50 %	4	9	5	13	3	8	0	1
LocationException		86 %		50 %	3	7	3	15	0	4	0	1
Location		92 %		50 %	2	8	2	14	0	6	0	1
DeclClass		69 %		42 %	14	24	25	73	4	11	0	1
DeclField		63 %		41 %	10	18	20	51	5	12	0	1
DeclMethod		71 %		36 %	17	24	21	68	3	9	0	1
DeclMethodAsm		53 %		31 %	13	16	22	39	5	8	0	1
StringLiteral		33 %		0 %	4	10	16	27	3	9	0	1
IfThenElse		72 %		0 %	3	10	18	55	1	8	0	1
ListExpr		9 %		0 %	3	4	8	9	1	2	0	1
AccessField		48 %		n/a	6	10	9	18	6	10	0	1
ReadInt		10 %		n/a	4	6	10	12	4	6	0	1
InstanceOf		57 %		n/a	3	7	10	19	3	7	0	1
NullLiteral		15 %		n/a	4	7	9	12	4	7	0	1
While		79 %		n/a	3	8	10	36	3	8	0	1
ReadFloat		33 %		n/a	3	6	8	12	3	6	0	1
Main		80 %		n/a	1	7	7	31	1	7	0	1
Program		88 %		n/a	3	9	5	37	3	9	0	1
AbstractUnaryExpr		60 %		n/a	1	5	5	14	1	5	0	1
EmptyMain		38 %		n/a	4	7	4	7	4	7	0	1
IntLiteral		90 %		n/a	1	9	2	16	1	9	0	1
NoOperation		63 %		n/a	1	6	2	7	1	6	0	1
AbstractInst		42 %		n/a	1	2	2	3	1	2	0	1
AbstractMethodBody		0 %		n/a	1	1	1	1	1	1	1	1
Or		97 %		n/a	1	3	1	16	1	3	0	1
And		97 %		n/a	1	3	1	16	1	3	0	1
ConvFloat		94 %		n/a	1	5	1	10	1	5	0	1
PrintIn		87 %		n/a	1	3	1	6	1	3	0	1
Print		71 %		n/a	1	2	1	3	1	2	0	1
AbstractDeclParam		60 %		n/a	1	2	1	2	1	2	0	1

Figure 3: Rapport Jacoco pour fr.ensimag.deca.tree

Atteindre 100% de couverture de code peut sembler idéal, mais ce n'est pas toujours la meilleure chose car cela demande un temps considérable pour réaliser tous les tests et ça peut conduire à des tests inutiles, surtout lorsqu'ils concernent des chemins de code rarement utilisés. De plus, une couverture élevée ne garantit pas nécessairement la qualité des tests ni leur capacité à détecter des bugs. On vise donc une couverture de minimum 70%, avec des tests qui ciblent les fonctionnalités critiques.

6 Annexe (scripts d'automatisation des tests)

```
cd "$(dirname "$0")"/../../.. || exit 1

PATH=./src/test/script/launchers:"$PATH"

test_lex_valide () {
    # $1 = premier argument.
    if test_lex "$1" 2>&1 | grep -q -e "$1:[0-9][0-9]*:"
    then
        echo "Echec inattendu pour test_lex sur $1."
        exit 1
    else
        echo "Succes attendu de test_lex sur $1."
    fi
}

test_lex_invalide () {
    # $1 = premier argument.
    if test_lex "$1" 2>&1 | grep -q -e "$1:[0-9][0-9]*:"
    then
        echo "Echec attendu pour test_lex sur $1."
    else
        echo "Succes inattendu de test_lex sur $1."
        exit 1
    fi
}

for cas_de_test in src/test/deca/syntax/valid/lexer/*.deca
do
    test_lex_valide "$cas_de_test"
done

for cas_de_test in src/test/deca/syntax/invalid/lexer/*.deca
do
    test_lex_invalide "$cas_de_test"
done
```

Figure 4: Script pour avance-lex.sh

```
cd "$(dirname "$0")"/../../.. || exit 1
PATH=./src/test/script/launchers:"$PATH"

test_synt_valide () {
    if test_synt "$1" 2>&1 | grep -q -e "$1:[0-9][0-9]*:"
    then
        echo "Echec inattendu pour test_synt sur $1."
        #exit 1
    else
        echo "Succes attendu de test_synt sur $1."
    fi
}

test_synt_invalide () {
    if test_synt "$1" 2>&1 | grep -q -e "$1:[0-9][0-9]*:"
    then
        echo "Echec attendu pour test_synt sur $1."
    else
        echo "Succes inattendu de test_synt sur $1."
        #exit 1
    fi
}

for cas_de_test in src/test/deca/syntax/valid/provided/*.deca
do
    test_synt_valide "$cas_de_test"
done

for cas_de_test in src/test/deca/syntax/valid/parser/*.deca
do
    test_synt_valide "$cas_de_test"
done

for cas_de_test in src/test/deca/syntax/invalid/provided/*.deca
do
    test_synt_invalide "$cas_de_test"
done

for cas_de_test in src/test/deca/syntax/invalid/parser/*.deca
do
    test_synt_invalide "$cas_de_test"
done
```

Figure 5: Script pour avance-synt.sh

```
cd "$(dirname "$0")"/../../.. || exit 1

PATH=./src/test/script/launchers:"$PATH"

test_cont_valide () {
    if test_context "$1" 2>&1 | grep -q -e "$1:[0-9][0-9]*:"
    then
        echo "Echec inattendu pour test_context sur $1."
        #exit 1
    else
        echo "Succes attendu de test_context sur $1."
    fi
}

test_cont_invalide () {
    if test_context "$1" 2>&1 | grep -q -e "$1:[0-9][0-9]*:"
    then
        echo "Echec attendu pour test_context sur $1."
    else
        echo "Succes inattendu de test_context sur $1."
        #exit 1
    fi
}

for cas_de_test in src/test/deca/context/valid/provided/*.deca
do
    test_cont_valide "$cas_de_test"
done

for cas_de_test in src/test/deca/context/valid/not_provided/*.deca
do
    test_cont_valide "$cas_de_test"
done

for cas_de_test in src/test/deca/context/invalid/provided/*.deca
do
    test_cont_invalide "$cas_de_test"
done

for cas_de_test in src/test/deca/context/invalid/not_provided/*.deca
do
    test_cont_invalide "$cas_de_test"
done
```

Figure 6: Script pour avance-context.sh

```
cd "$(dirname "$0")"/../../.. || exit 1
PATH=./src/test/script/launchers:./src/main/bin:"$PATH"
TEST_DIR="src/test/deca/codegen/valid/not_provided"
EXPECTED_DIR="$TEST_DIR/expected"

test_gencode_valide () {
    local test_file="$1"
    local base_name=$(basename "$test_file" .deca)
    local ass_file="${TEST_DIR}/${base_name}.ass"
    local expected_file="${EXPECTED_DIR}/${base_name}.expected"
    local output_file="${TEST_DIR}/${base_name}.out"

    rm -f "$ass_file" "$output_file" 2>/dev/null

    echo "Compilation de $test_file..."
    decac "$test_file" || { echo "Erreur de compilation pour $test_file"; exit 1; }

    if [ ! -f "$ass_file" ]; then
        echo "Fichier .ass non généré pour $test_file"
        exit 1
    fi

    echo "Exécution de $ass_file avec ima..."
    ima "$ass_file" > "$output_file" || { echo "Erreur d'exécution avec ima pour $test_file"; exit 1; }

    if [ ! -f "$expected_file" ]; then
        echo "Fichier attendu (.expected) manquant pour $test_file"
        exit 1
    fi
    if diff -q "$output_file" "$expected_file" >/dev/null; then
        echo "Test réussi pour $test_file"
    else
        echo "Test échoué pour $test_file"
        echo "Résultat obtenu :"
        cat "$output_file"
        echo "Résultat attendu :"
        cat "$expected_file"
        exit 1
    fi
}

for test_file in "$TEST_DIR"/*.deca; do
    test_gencode_valide "$test_file"
done
echo "Tous les tests de gencode ont été exécutés avec succès."
```

Figure 7: Script pour avance-gencode.sh